

```

# -*- coding: utf-8 -*-
"""
Created on Sun Sep  2 17:22:25 2018

@author: jfowl


$$x\ddot{ } + 7x = 5\cos(2t)$$


Analytical Solution:

$$x = -35/53e^{(-7t)} + 35/53\cos(2t) + 10/53\sin(2t)$$

"""

import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import odeint

nt = 1000
tstart = 0
tend = 20

tvec = np.linspace(tstart, tend, nt)

def analytic_sol(t):
    c1 = -35/53
    c2 = 35/53
    c3 = 10/53
    first = c1*np.exp(-7*t)
    second = c2*np.cos(2*t)
    third = c3*np.sin(2*t)
    x = first + second + third
    return x

def ode_model(x, t):
    c = 7
    c1 = 5
    c2 = 2
    dxdt = -c*x + c1*np.cos(c2*t)
    return dxdt

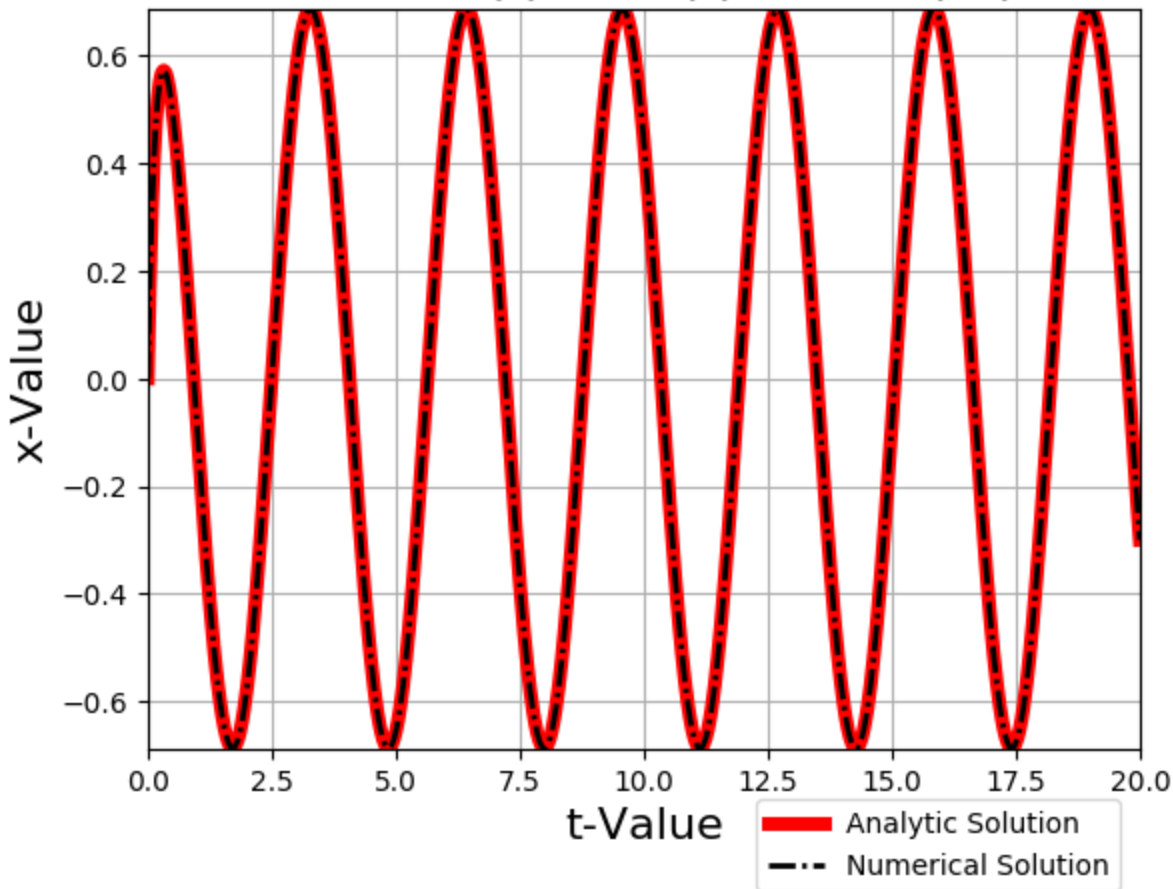
x0 = 0
x = odeint(ode_model, x0, tvec)

fig = plt.figure()
ax = fig.add_subplot(111)
ax.plot(tvec, [analytic_sol(t) for t in tvec], 'r', linewidth=5,
        label='Analytic Solution')
ax.plot(tvec, x, '-.k', linewidth=2, label='Numerical Solution')
plt.xlim([min(tvec), max(tvec)])
plt.ylim([min(x), max(x)])
handles, labels = ax.get_legend_handles_labels()
lgnd = ax.legend(handles, labels, loc='upper left',
                 bbox_to_anchor=(0.6, -0.05))
plt.xlabel('t-Value', fontsize=16)

```

```
plt.ylabel('x-Value', fontsize=16)
plt.title('Plot of  $x'(t) + 7x(t) = 5\cos(2t)$ ', fontsize=18)
plt.grid(b=True, which='both')
fig.savefig('ME426_HW3_20a', bbox_extra_artists=(lgnd,), bbox_inches='tight')
plt.show()
```

Plot of  $x'(t) + 7x(t) = 5\cos(2t)$



```

# -*- coding: utf-8 -*-
"""
Created on Sun Sep  2 17:23:16 2018

@author: jfowl

xdbldot + 6xdot + 8x = 5sin(3t)

with analytic solution:
    x = 9/65e^-2t + 9/65e^-4t - 1/65sin(3t) - 18/65cos(3t)
"""

import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import odeint

nt = 1000
tstart = 0
tend = 20

tvec = np.linspace(tstart, tend, nt)

def analytic_sol(t):
    c = 75/130
    c1 = -39/130
    c2 = -2/130
    c3 = -36/130
    first = c*np.exp(-2*t) + c1*np.exp(-4*t)
    second = c2*np.sin(3*t)
    third = c3*np.cos(3*t)
    x = first + second + third
    return x

def ode_model(X, t):
    xdot1 = X[1]
    xdot2 = -6*X[1] - 8*X[0] + 5*np.sin(3*t)
    return [xdot1, xdot2]

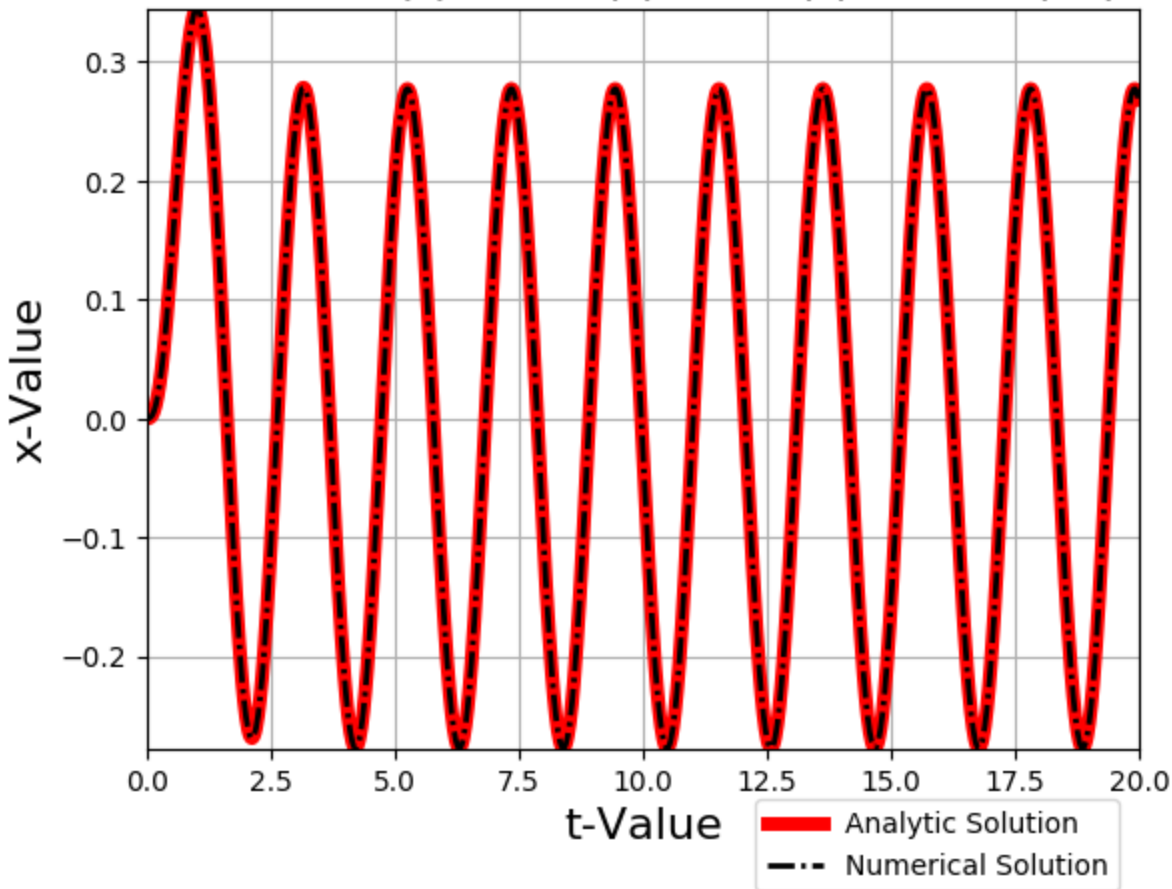
x0 = [0, 0]
x = odeint(ode_model, x0, tvec)

fig = plt.figure()
ax = fig.add_subplot(111)
ax.plot(tvec, [analytic_sol(t) for t in tvec], 'r', linewidth=5,
        label='Analytic Solution')
ax.plot(tvec, x[:, 0], '-.k', linewidth=2, label='Numerical Solution')
plt.xlim([min(tvec), max(tvec)])
plt.ylim([min(x[:, 0]), max(x[:, 0])])
handles, labels = ax.get_legend_handles_labels()
lgnd = ax.legend(handles, labels, loc='upper left',
                 bbox_to_anchor=(0.6, -0.05))
plt.xlabel('t-Value', fontsize=16)
plt.ylabel('x-Value', fontsize=16)

```

```
plt.title('Plot of  $x''(t) + 6x'(t) + 8x(t) = 5\sin(3t)$ ', fontsize=18)
plt.grid(b=True, which='both')
fig.savefig('ME426_HW3_20b', bbox_extra_artists=(lgnd,), bbox_inches='tight')
plt.show()
```

Plot of  $x''(t) + 6x'(t) + 8x(t) = 5\sin(3t)$



```

# -*- coding: utf-8 -*-
"""
Created on Sun Sep  2 17:23:27 2018

@author: jfowl


$$x''(t) + 8x'(t) + 25x(t) = 10u(t)$$

with analytic solution:

$$x = -e^{-4t}(10/25\cos(3t) + 8/15\sin(3t)) + 10/25$$

"""

import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import odeint

nt = 1000
tstart = 0
tend = 20

tvec = np.linspace(tstart, tend, nt)

def analytic_sol(t):
    c = 10/25
    c1 = 8/15
    first = -np.exp(-4*t)
    second = c*np.cos(3*t)
    third = c1*np.sin(3*t)
    x = first*(second + third) + c
    return x

def ode_model(X, t):
    xdot1 = X[1]
    xdot2 = -8*X[1] - 25*X[0] + 10
    return [xdot1, xdot2]

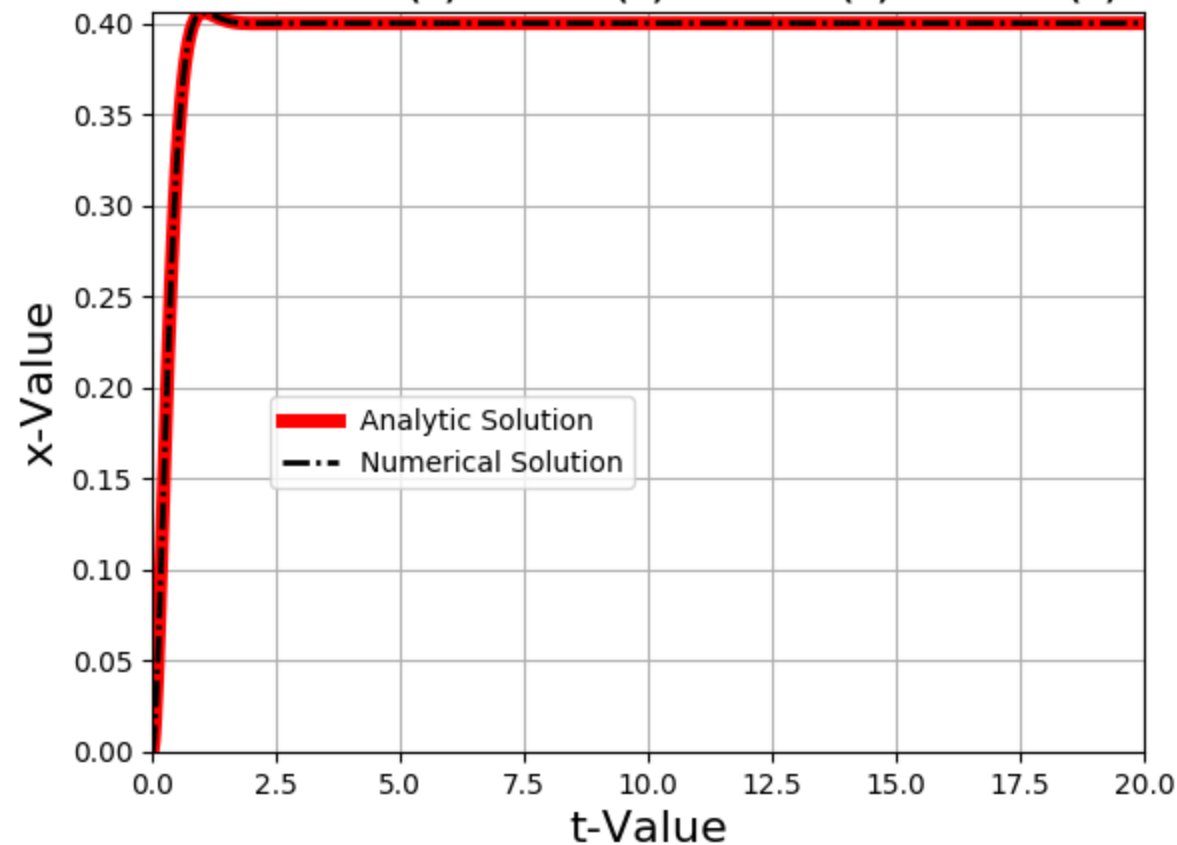
x0 = [0, 0]
x = odeint(ode_model, x0, tvec)

fig = plt.figure()
ax = fig.add_subplot(111)
ax.plot(tvec, [analytic_sol(t) for t in tvec], 'r', linewidth=5,
        label='Analytic Solution')
ax.plot(tvec, x[:, 0], '-.k', linewidth=2, label='Numerical Solution')
plt.xlim([min(tvec), max(tvec)])
plt.ylim([min(x[:, 0]), max(x[:, 0])])
handles, labels = ax.get_legend_handles_labels()
lgnd = ax.legend(handles, labels, loc='best',
                 bbox_to_anchor=(0.5, 0.5))
plt.xlabel('t-Value', fontsize=16)
plt.ylabel('x-Value', fontsize=16)
plt.title('Plot of  $x''(t) + 8x'(t) + 25x(t) = 10u(t)$ ', fontsize=18)
plt.grid(b=True, which='both')
fig.savefig('ME426_HW3_20c', bbox_extra_artists=(lgnd,), bbox_inches='tight')

```

```
plt.show()
```

Plot of  $x''(t) + 8x'(t) + 25x(t) = 10u(t)$



```

# -*- coding: utf-8 -*-
"""
Created on Sun Sep  2 17:23:37 2018

@author: jfowl

 $x''(t) + 2x'(t) + 2x = \sin(2t)$ 
with initial conditions:
     $x(0) = 2$ 
     $x'(0) = -3$ 

with analytic solution:
     $x = e^{-t}(-3/5\sin(t) + 11/5\cos(t)) - 1/10\sin(2t) - 1/5\cos(2t)$ 
"""

import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import odeint

nt = 1000
tstart = 0
tend = 20

tvec = np.linspace(tstart, tend, nt)

def analytic_sol(t):
    c = -3/5
    c1 = 11/5
    c2 = -1/10
    c3 = -1/5
    first = np.exp(-t)
    second = c*np.sin(t)
    third = c1*np.cos(t)
    fourth = c2*np.sin(2*t)
    fifth = c3*np.cos(2*t)
    x = first*(second + third) + fourth + fifth
    return x

def ode_model(X, t):
    xdot1 = X[1]
    xdot2 = -2*X[1] - 2*X[0] + np.sin(2*t)
    return [xdot1, xdot2]

x0 = [2, -3]
x = odeint(ode_model, x0, tvec)

fig = plt.figure()
ax = fig.add_subplot(111)
ax.plot(tvec, [analytic_sol(t) for t in tvec], 'r', linewidth=5,
        label='Analytic Solution')
ax.plot(tvec, x[:, 0], '-.k', linewidth=2, label='Numerical Solution')
plt.xlim([min(tvec), max(tvec)])
plt.ylim([min(x[:, 0]), max(x[:, 0])])

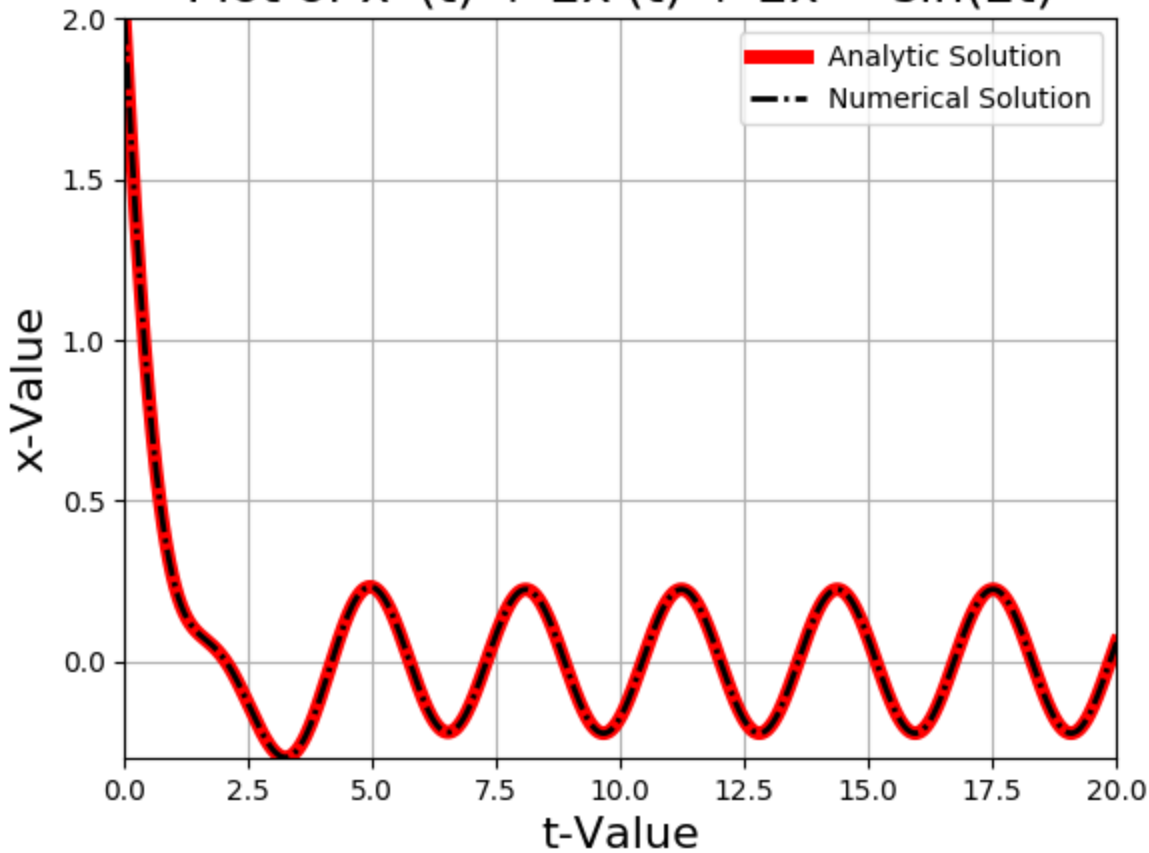
```

```

handles, labels = ax.get_legend_handles_labels()
lgnd = ax.legend(handles, labels, loc='upper right',
                 bbox_to_anchor=(1, 1))
plt.xlabel('t-Value', fontsize=16)
plt.ylabel('x-Value', fontsize=16)
plt.title('Plot of  $x''(t) + 2x'(t) + 2x = \sin(2t)$ ', fontsize=18)
plt.grid(b=True, which='both')
fig.savefig('ME426_HW3_21a', bbox_extra_artists=(lgnd,), bbox_inches='tight')
plt.show()

```

Plot of  $x''(t) + 2x'(t) + 2x = \sin(2t)$



```

# -*- coding: utf-8 -*-
"""
Created on Sun Sep  2 17:23:58 2018

@author: jfowl


$$x''(t) + 2x'(t) + x = 5e^{-2t} + t$$

with initial conditions:
    
$$x(0) = 2$$

    
$$x'(0) = 1$$


with analytic solution:
    
$$x = -e^{-t} + 9te^{-t} + 5e^{-2t} + t - 2$$

"""

import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import odeint

nt = 1000
tstart = 0
tend = 20

tvec = np.linspace(tstart, tend, nt)

def analytic_sol(t):
    first = -np.exp(-t)
    second = 9*t*np.exp(-t)
    third = 5*np.exp(-2*t)
    fourth = t
    fifth = -2
    x = first + second + third + fourth + fifth
    return x

def ode_model(X, t):
    xdot1 = X[1]
    xdot2 = -2*X[1] - X[0] + 5*np.exp(-2*t) + t
    return [xdot1, xdot2]

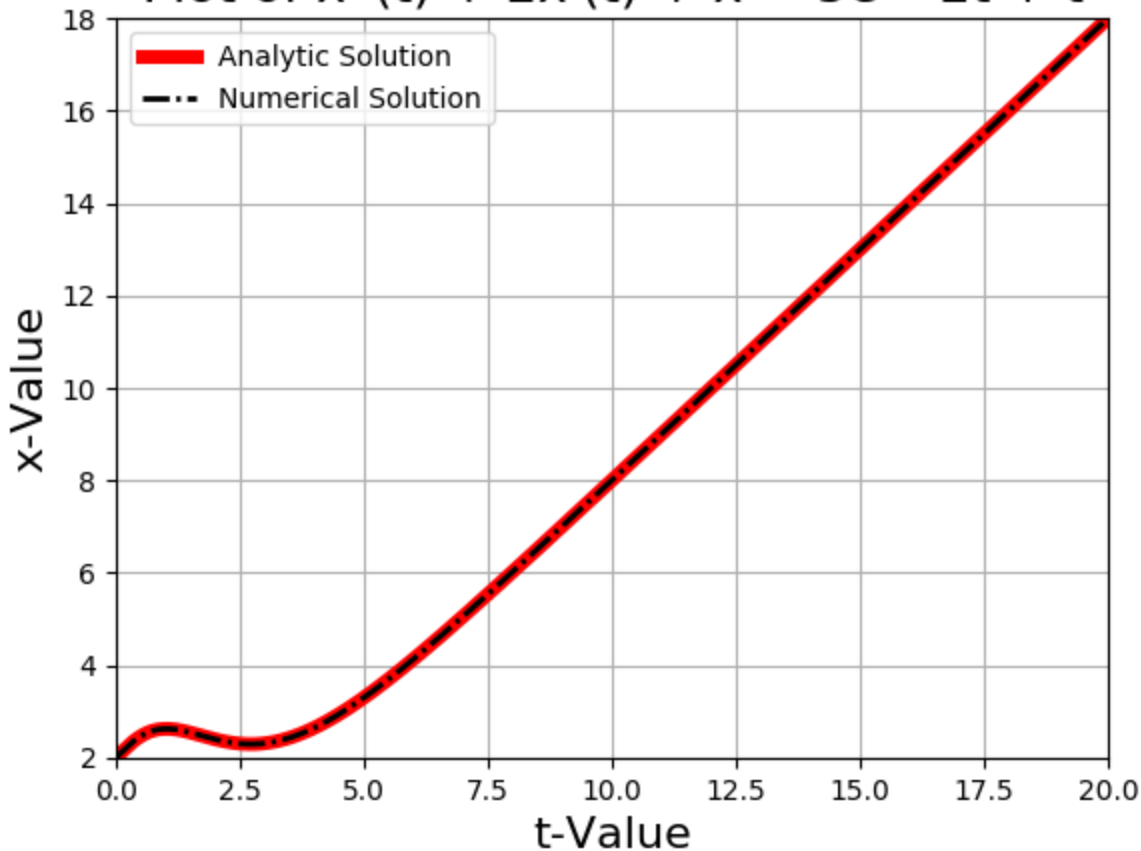
x0 = [2, 1]
x = odeint(ode_model, x0, tvec)

fig = plt.figure()
ax = fig.add_subplot(111)
ax.plot(tvec, [analytic_sol(t) for t in tvec], 'r', linewidth=5,
        label='Analytic Solution')
ax.plot(tvec, x[:, 0], '-.k', linewidth=2, label='Numerical Solution')
plt.xlim([min(tvec), max(tvec)])
plt.ylim([min(x[:, 0]), max(x[:, 0])])
handles, labels = ax.get_legend_handles_labels()
lgnd = ax.legend(handles, labels, loc='upper left',
                 bbox_to_anchor=(0, 1))
plt.xlabel('t-Value', fontsize=16)

```

```
plt.ylabel('x-Value', fontsize=16)
plt.title('Plot of  $x''(t) + 2x'(t) + x = 5e^{-2t} + t$ ', fontsize=18)
plt.grid(b=True, which='both')
fig.savefig('ME426_HW3_21b', bbox_extra_artists=(lgnd,), bbox_inches='tight')
plt.show()
```

Plot of  $x''(t) + 2x'(t) + x = 5e^{-2t} + t$



```

# -*- coding: utf-8 -*-
"""
Created on Sun Sep  2 17:24:06 2018

@author: jfowl

 $x''(t) + 4x = t^2$ 
with initial conditions:
     $x(0) = 1$ 
     $x'(0) = 2$ 

with analytic solution:
     $x = 9/8\cos(2t) + \sin(2t) + 1/4t^2 - 1/8$ 
"""

import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import odeint

nt = 1000
tstart = 0
tend = 20

tvec = np.linspace(tstart, tend, nt)

def analytic_sol(t):
    c = 9/8
    c1 = 1/4
    c2 = -1/8
    first = c*np.cos(2*t)
    second = np.sin(2*t)
    third = c1*t**2
    fourth = c2
    x = first + second + third + fourth
    return x

def ode_model(X, t):
    xdot1 = X[1]
    xdot2 = -4*X[0] + t**2
    return [xdot1, xdot2]

x0 = [1, 2]
x = odeint(ode_model, x0, tvec)

fig = plt.figure()
ax = fig.add_subplot(111)
ax.plot(tvec, [analytic_sol(t) for t in tvec], 'r', linewidth=5,
        label='Analytic Solution')
ax.plot(tvec, x[:, 0], '-.k', linewidth=2, label='Numerical Solution')
plt.xlim([min(tvec), max(tvec)])
plt.ylim([min(x[:, 0]), max(x[:, 0])])
handles, labels = ax.get_legend_handles_labels()
lgnd = ax.legend(handles, labels, loc='upper left',

```

```
        bbox_to_anchor=(0, 1))
plt.xlabel('t-Value', fontsize=16)
plt.ylabel('x-Value', fontsize=16)
plt.title('Plot of  $x(t) + 4x = t^2$ ', fontsize=18)
plt.grid(b=True, which='both')
fig.savefig('ME426_HW3_21c', bbox_extra_artists=(lgnd,), bbox_inches='tight')
plt.show()
```

Plot of  $x''(t) + 4x = t^2$

